Memory Consistency

Some slides from:
UT Austin CS378h (Chris Rossbach)
Stanford CS149 (Kunle Olukotun and Kayvon Fatahalian)

Administrivia

- HW6 (Cuckoo hash) due today
- HW7 (Cache Coherence) section tomorrow
- Quiz Friday

Agenda

- Memory Consistency

Who should care about Memory Consistency?

- Low-level systems programmers
- Compiler developers
- Anyone who wants to use Atomics
- Anyone who wants to use a programming language with multithreading
  - Languages have their own memory consistency model!

Two Hard Things

There are only two hard things in Computer Science: cache invalidation and naming things.

– Phil Karlton

Memory Consistency

- Can one core reorder memory operations?
- Do all cores have to see the same order of memory operations?

Memory Consistency Question

Initially, A = B = 0

Core 0:

```
A = 1
print(B)
```

Core 1:

```
B = 1
print(A)
```

What can be printed?
“01”?
“10”?
“11”?
“00”?

Memory Consistency Affects:

- Programmability: How programmers reason about program behavior
  - What behavior is allowed?

- Performance: limits HW optimizations that can be used
  - Reordering operations to hide latency

Various consistency models

- Linearizability (Strict Consistency)
- Sequential Consistency
- Causal Consistency
- Total Store Order — x86
- Processor Consistency
- Partial Store Order
- Weak Consistency — ARM

Two axes, so this is only a partial ordering!

Linearizability

- Can one core reorder memory operations?
  - No!
- Do all cores have to see the same order of memory operations?
  - Yes! And it must match real-time order.

Weak Consistency

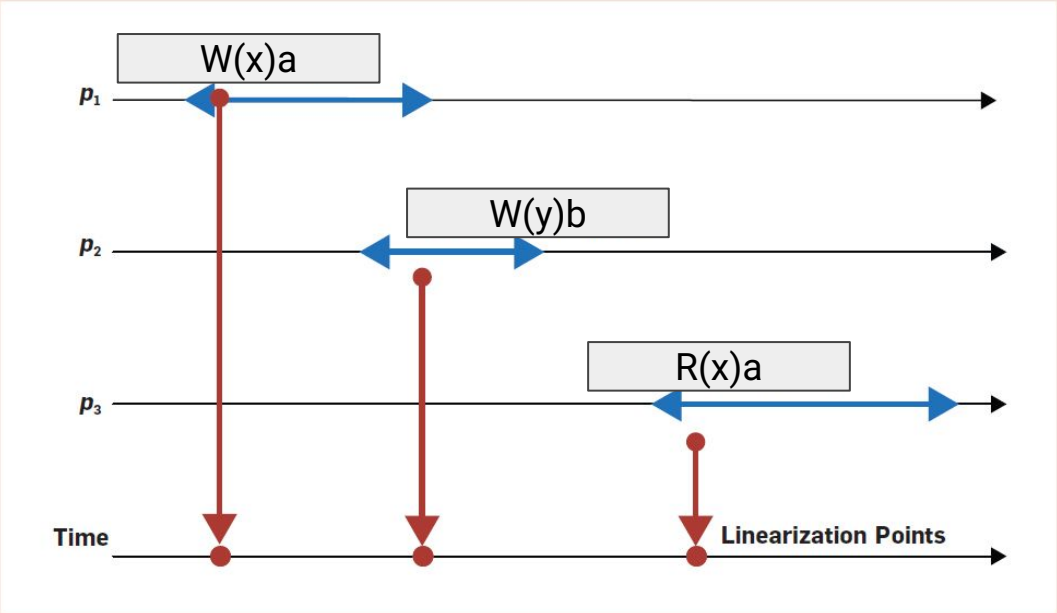
- Can one core reorder memory operations?
  - Yes!
- Do all cores have to see the same order of memory operations?
  - No!

Before we start going into consistency models…

- Memory consistency
  - Defines the behavior of reads & writes to different memory locations
  - In what order do these reads & writes happen? Real-time ordering is very difficult!
- Memory coherence
  - Defines the behavior of reads & writes to one memory location
  - Part of an implementation of a consistency model

The Gold Standard: Linearizability

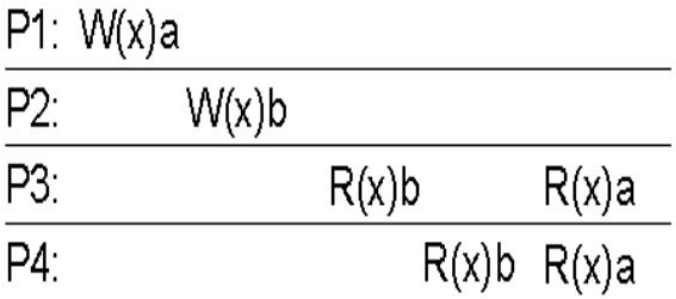
- The only consistent memory is one that respects the real-time ordering of operations
- As if memory is instant

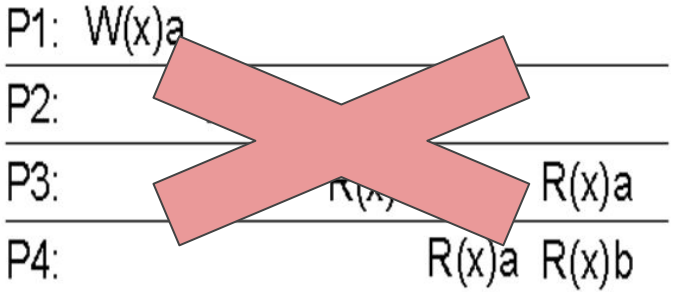


What Most Programmers Expect: Sequential Consistency

- Memory operations can be observed in non-real-time orderings
- Each processor respects program order
- All cores still see the same order

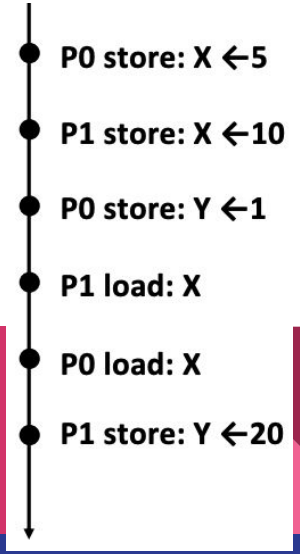
Together, these two mean that the system still behaves like there's a "single shared memory"





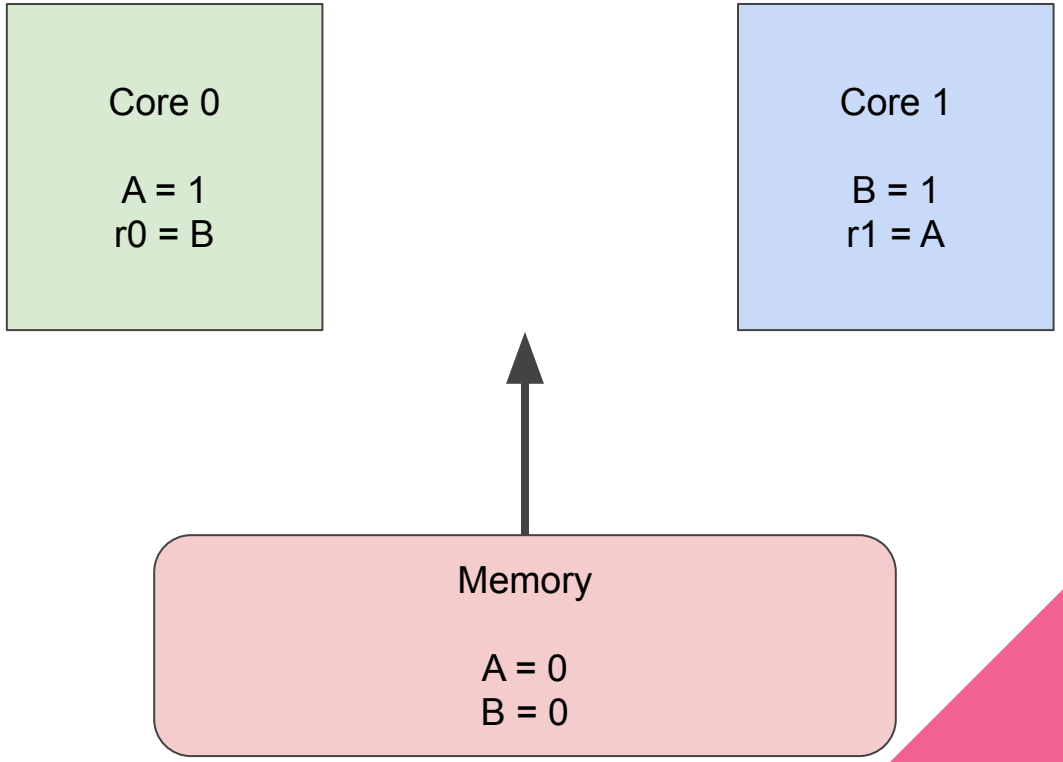
Sequential Consistency

- Lamport 1976 (Turing Award 2013)
- There is a chronology of all memory operations consistent with core-observed values
- Switch metaphor:
  - Memory chooses a core, performs one memory op, repeat.

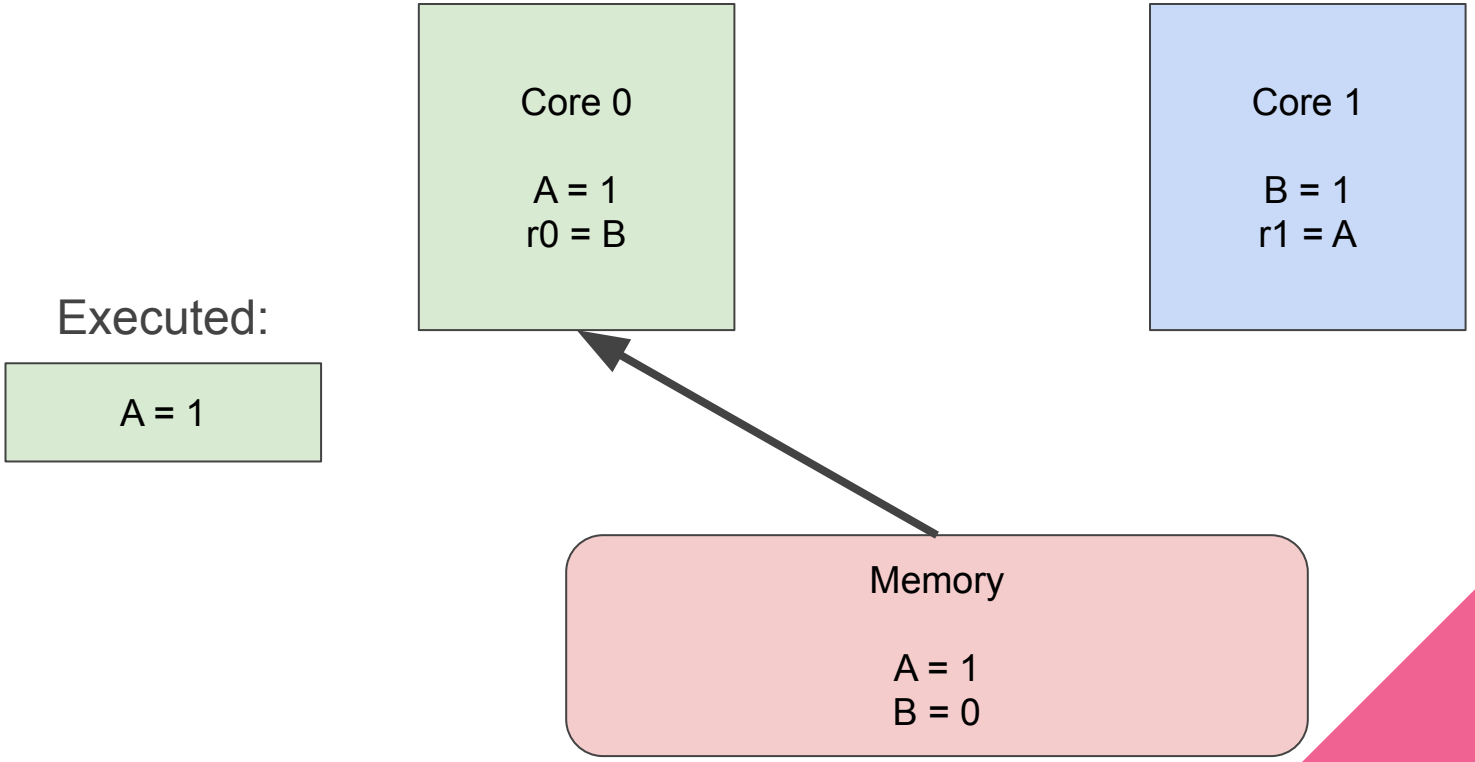


Sequential Consistency: Switch

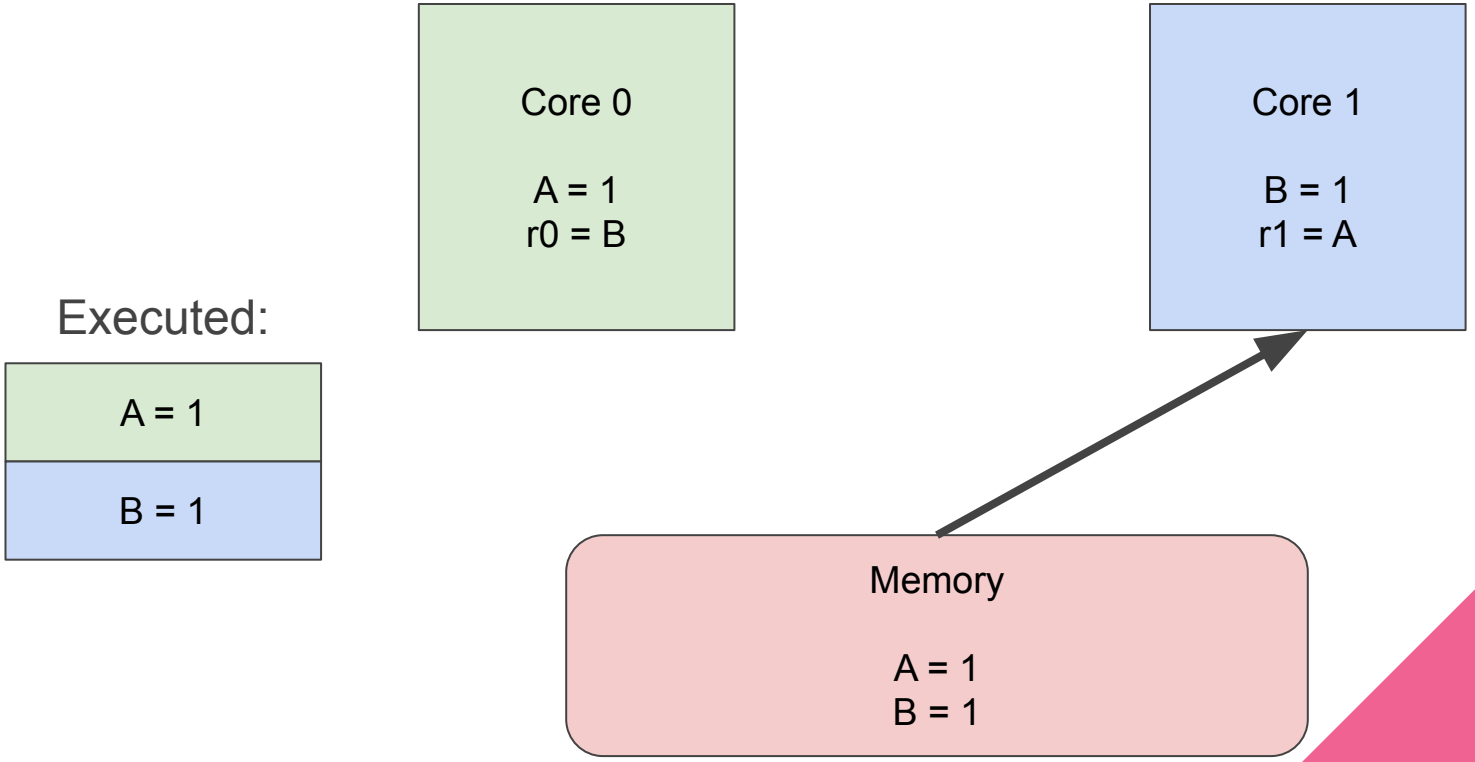
Executed:



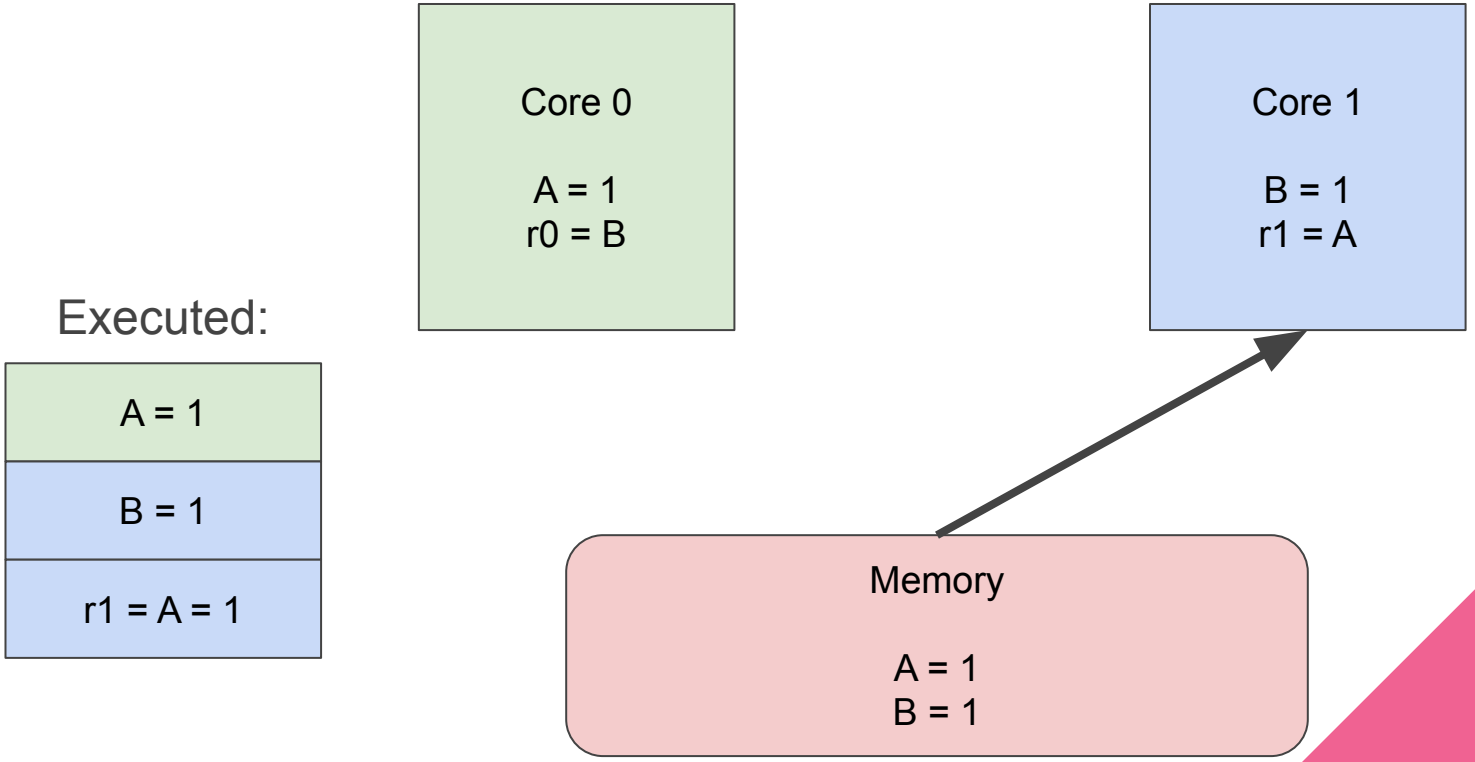
Sequential Consistency: Switch



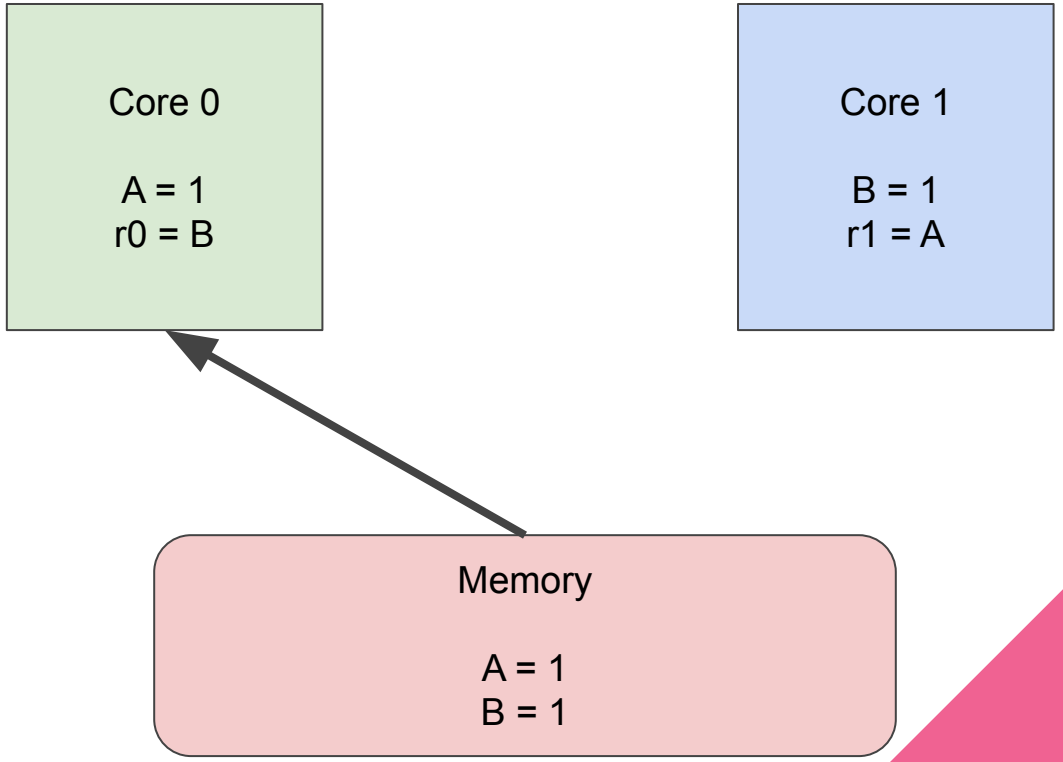
Sequential Consistency: Switch



Sequential Consistency: Switch



Sequential Consistency: Switch



Executed:

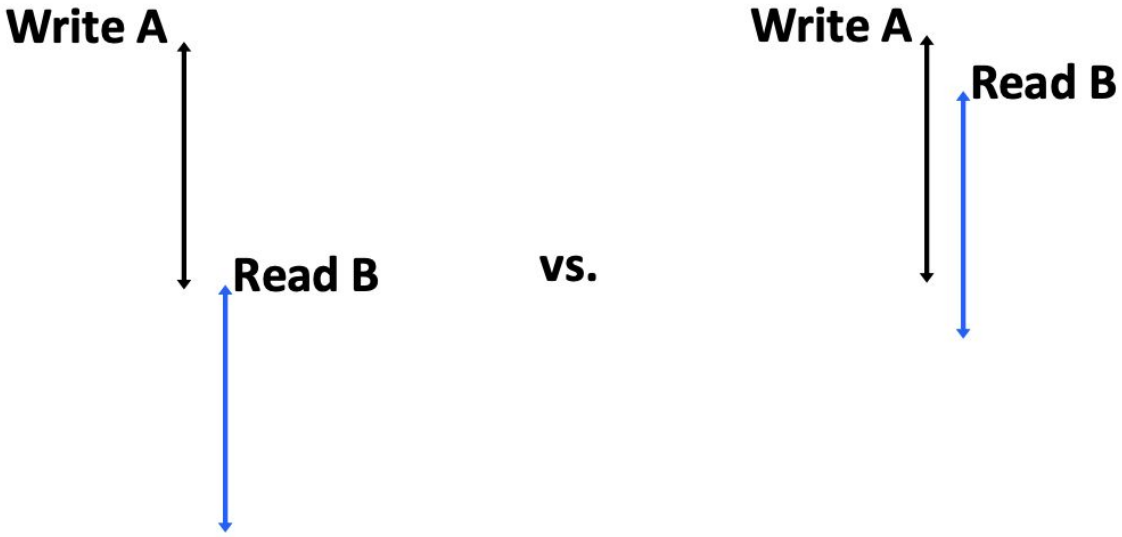
A = 1
B = 1
r1 = A = 1
r0 = B = 1

Sequential Consistency

- Can one core reorder memory operations?
  - No!
- Do all cores have to see the same order of memory operations?
  - Yes, but it does not have to match real-time order.

Sequential Consistency is Slow

We aren’t allowed to overlap reads and writes at all!



Relaxing Memory Operation Ordering

Cores now can overlap & reorder some memory operations.

How do we maintain data-race-freeness in our programs? Memory fences!

Memory Fences

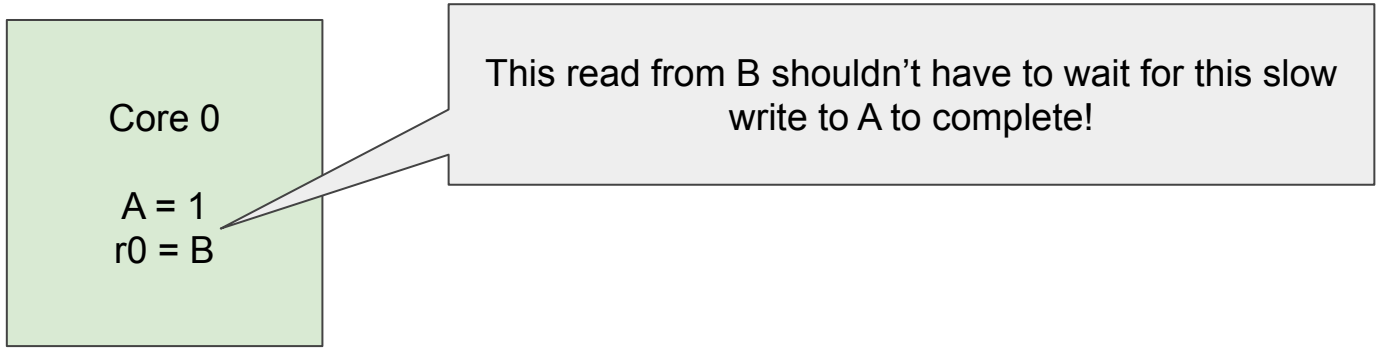
Ban CPUs from reordering memory operations across them

In other words, all memory operations must be done to pass through the fence!

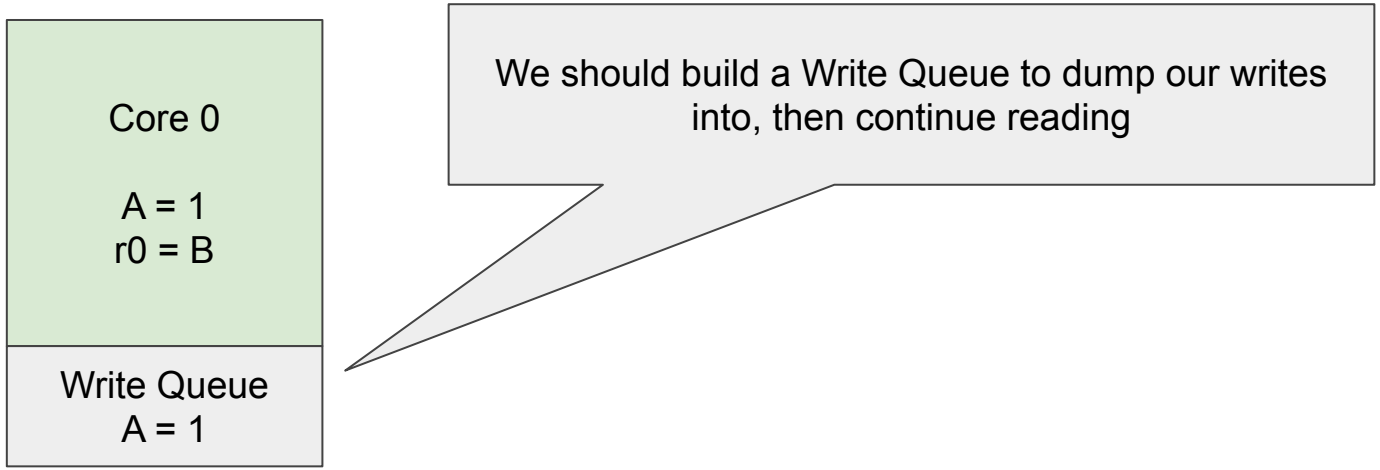
Different types of orderings

- W -> R: Write to x must commit before subsequent read from y
- R -> R: Read from x must commit before subsequent read from y
- R -> W: Read from x must commit before subsequent write to y
- W -> W: Write to x must commit before subsequent write to y

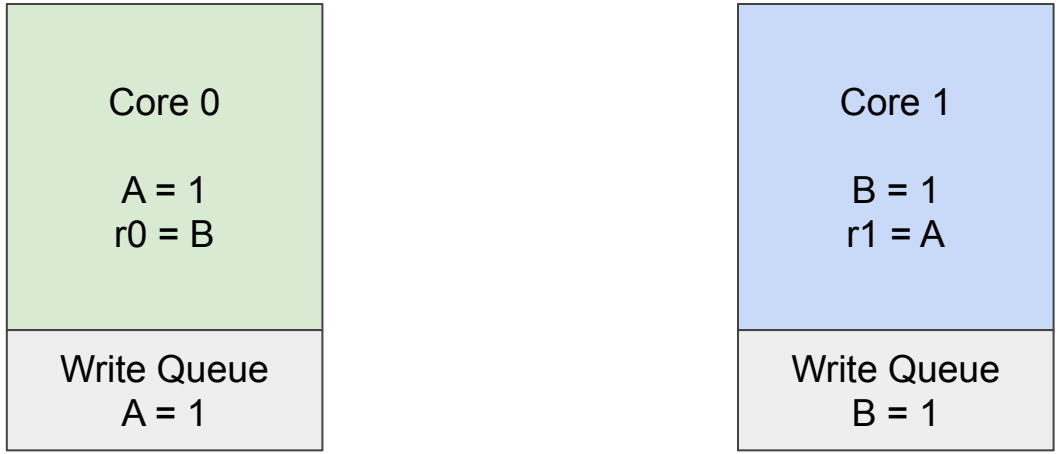
Hardware Optimization Request



Hardware Optimization Request



Hardware Optimization Request



Total Store Order

= Sequential Consistency + Write queues

- Allows us to see the “00” result from earlier
- Proper fencing (hardware synchronization) is required in some cases
- But provides speedups!

~~W -> R~~
R -> R
R -> W
W -> W

More HW Optimizations!

- Let’s add a load queue too!
- Now, we’ve removed all orderings!
- Let’s find bugs in these ultra-relaxed models.

Relaxed Consistency: Find the bugs!

Initially, flag = UNSET, and data1 and data2 are 0.

Core C1	Core C2
S1: data1 = NEW;	
S2: data2 = NEW;	
S3: flag = SET;	L1: r1 = flag;
	B1: if (r1≠ SET) goto L1;
	L2: r2 = data1;
	L3: r3 = data2;

Relaxed Consistency Models

- Can one core reorder memory operations?
  - Yep, depending on the model (TSO, PSO)
- Do all cores have to see the same order of memory operations?
  - Yes, still.

Weak Consistency Models

- Cores can see different orders from each other!

P1:	W(x)a				
P2:		W(x)b			
P3:			R(x)b		R(x)a
P4:				R(x)a	R(x)b

Causal Consistency

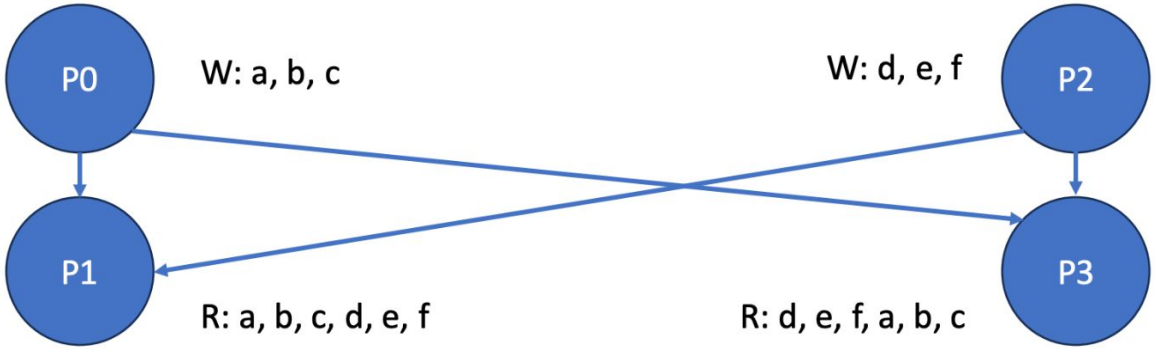
- Causal relationships must be respected (cannot reorder R->W)
- Non-causal relationships can be weakly ordered

P1	W(x) 1			
P2	R(x) 1	W(y) 2		
P3			R(x) 1	R(y) 2

Processor Consistent

- Only requirement is that writes cannot be reordered (W->W)

P1	W(x) 1	W(x) 2				
P2			R(x) 2	W(y) 3		
P3					R(y) 3	R(x) 1



1. P1 sees P0's writes in P0 order
2. P1 sees P2's writes in P2 order
3. Same for P3
4. P3 may see different interleavings of P0, P2 writes than P1 observes

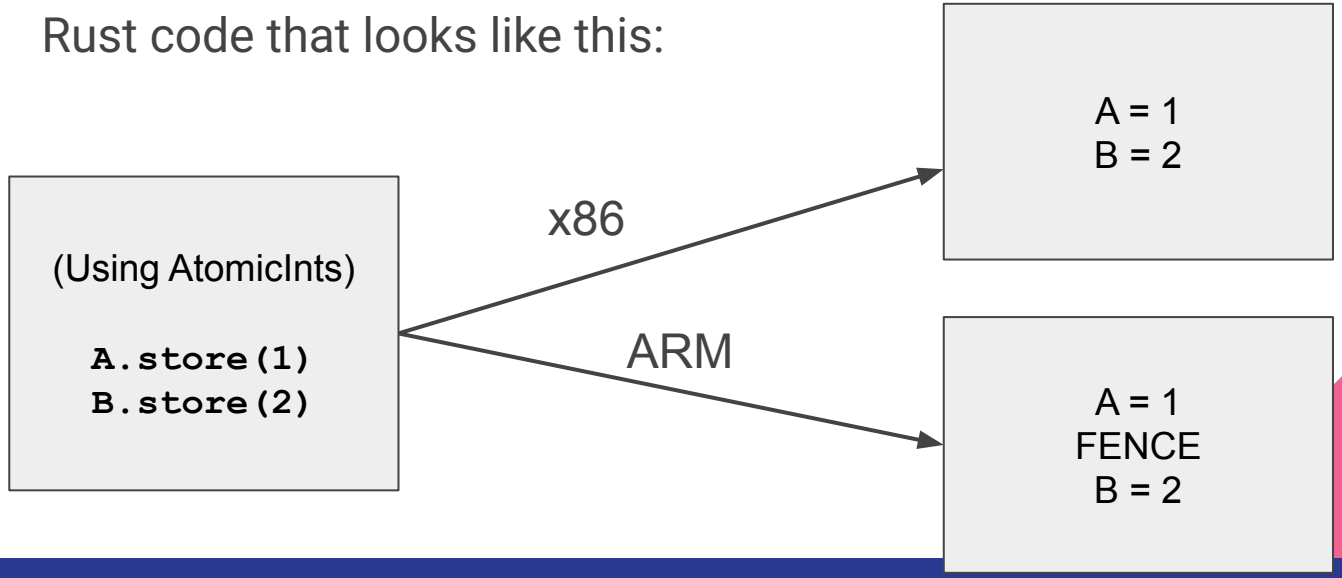
What consistency is this?

“Eventual Consistency”

P1	W(x) 1	W(x) 2	W(y) 3			
P2				R(y) 3	R(x) 1	R(x) 2

Summary

- Weak consistency -> More performance, but more potential bugs
- Notice: these are all data races!
- But PL-level data races != HW-level data races.
- Rust code that looks like this:



Language Level Memory Models

- C, C++, Java, and Rust all guarantee Sequential Consistency behavior for Data-Race-Free programs (“SC for DRF”)
  - Compilers will insert the necessary fences to cope with the hardware memory model
- No guarantees for programs with data races!

Atomics

- Every atomic operation comes with a “memory order”: an optional memory fence to be issued with that atomic operation
  - Can ensure that all non-atomic writes have committed before setting AtomicBool to True

- SeqCst (Everything around me must stay on either side of me, full fence)
- Acquire (things after me must begin after I do)
- Release (things before me must commit before I do)
- Relaxed (I don’t care)